



Portfolio Phase 2

Internationale Hochschule Duales Studium

Studiengang: Informatik

RideAndPark - Erarbeitungsphase

Lukas Penner

Matrikelnummer: 4234713

Elsternhorst 13, 35415 Pohlheim

Betreuer/in: Armands Strazds

Abgabe: 07.05.2026

Inhaltsverzeichnis

1.	Umsetzung des Projekts	1
2.	Fortschritt und Projektorganisation	1
3.	Datenmodell und Verarbeitung	3
4.	Herausforderungen und Probleme	3
5.	Architektur	4
6.	Zentrale Entscheidungen	5
7.	Reflexion	6
8.	Zwischenstand und Ausblick	6
9.	Fazit	7

1. Umsetzung des Projekts

In der zweiten Projektphase wurde die in Phase 1 geplante Systemarchitektur praktisch umgesetzt. Ziel war die Entwicklung einer Full-Stack-Anwendung zur Verarbeitung und Darstellung von Parkplatzdaten.

Der Schwerpunkt lag auf dem Aufbau eines eigenen Backends mit Node.js und Express. Die externe Parkplatzquelle wird nicht direkt vom Frontend aufgerufen, sondern über das Backend verarbeitet. Dadurch können Daten serverseitig transformiert, gefiltert und kontrolliert bereitgestellt werden.

Das Backend folgt einer mehrschichtigen Architektur:

- Routes definieren die API-Endpunkte
- Controller validieren Anfragen
- Services enthalten die fachliche Logik
- Ein Transformer vereinheitlicht externe Rohdaten in ein internes Datenmodell

Die Anwendung folgt damit einem API-first-Ansatz. Das Frontend kommuniziert ausschließlich mit der eigenen REST-API und bleibt unabhängig von der Struktur der externen Datenquelle.

Wichtige API-Endpunkte sind:

Tabelle 1: API-Endpunkte

Methode	Endpunkt	Zweck
GET	/api/parkings	Parkplatzliste
GET	/api/parkings/:id	Einzelnen Parkplatz abrufen
POST	/api/parkings/refresh	Daten aktualisieren
GET	/api/geocode	Zieladresse geocodieren

Quelle: Eigene Darstellung

Zusätzlich wurden erste Mechanismen für Caching, Fehlerbehandlung und Fallback-Daten umgesetzt.

2. Fortschritt und Projektorganisation

Die Entwicklung wurde über ein Kanban-Board organisiert. Die Aufgaben wurden in die Bereiche Backlog, Zu erledigen, In Bearbeitung und Abgeschlossen unterteilt.

Abbildung 1: Kanban-Board zur Projektorganisation



Quelle: Eigene Darstellung aus Miro

Bereits abgeschlossen wurden zentrale Backend-Aufgaben wie:

- Aufsetzen von Node.js
- Definition der Architektur

- API-Anbindung
- Datenverarbeitung
- Filterlogik

Aktuell befinden sich unter anderem die Stabilisierung der Performance sowie das React-Frontend in Bearbeitung. Weitere Aufgaben wie Responsive Design, Kartenintegration, Tests und Performance-Optimierung befinden sich noch im Backlog.

Die Kanban-Struktur ermöglicht eine transparente Nachverfolgung des Fortschritts und unterstützt eine iterative Weiterentwicklung des Projekts.

3. Datenmodell und Verarbeitung

Ein zentraler Bestandteil der zweiten Phase war die Einführung eines internen Datenmodells. Externe Parkplatzdaten werden dabei in eine einheitliche Struktur überführt.

Das Modell enthält unter anderem:

- Parkplatz-ID
- Name
- Koordinaten
- freie Stellplätze
- Gesamtkapazität
- Status
- Zeitstempel

Der Transformationsprozess vereinheitlicht unterschiedliche externe Datenstrukturen und reduziert die Komplexität im Frontend. Dadurch kann die Anwendung unabhängig von Änderungen der externen API weiterarbeiten.

4. Herausforderungen und Probleme

Die größte Herausforderung liegt aktuell im Bereich der Performance. Ursache dafür sind vor allem:

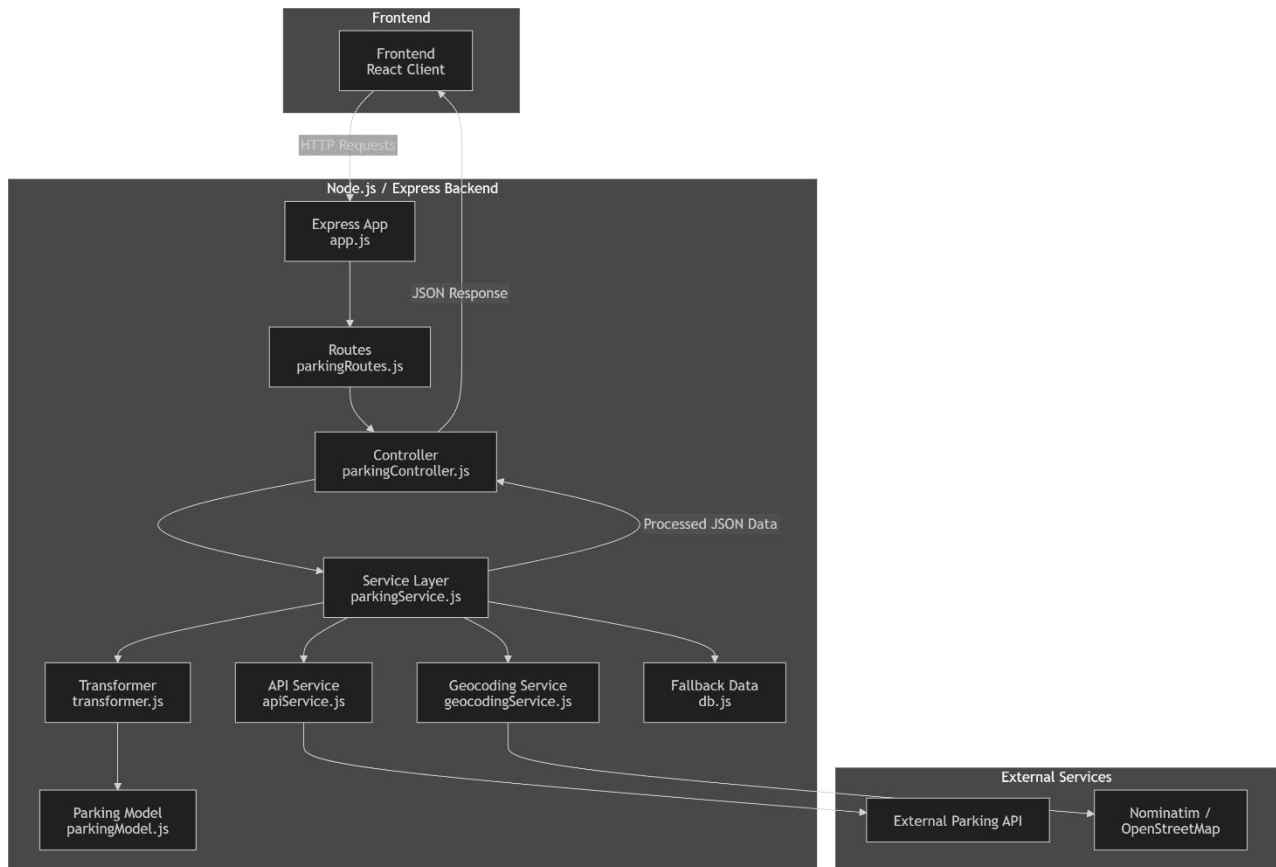
- externe API-Aufrufe,
- die Transformation heterogener Daten,
- serverseitige Filterungen,
- sowie die Verarbeitung von Echtzeitdaten.

Zusätzlich besteht eine starke Abhängigkeit von der externen Parkplatz-API. Langsame Antwortzeiten oder Ausfälle wirken sich direkt auf die Anwendung aus.

Zur Verbesserung wurden bereits Timeouts, Retry-Strategien und Fallback-Daten implementiert. Dennoch besteht weiterhin Optimierungsbedarf, insbesondere bei der Transformationslogik und der allgemeinen Verarbeitungseffizienz.

5. Architektur

Abbildung 2: Architektur im Backend



Quelle: Eigene Darstellung

Die dargestellte Architektur zeigt den vollständigen Datenfluss zwischen Frontend, Backend und externen Diensten.

Das mit React entwickelte Frontend kommuniziert ausschließlich über REST-Schnittstellen mit dem Express.js-Backend. Dadurch wird verhindert, dass das Frontend direkt auf externe APIs zugreifen muss.

Im Zentrum des Diagramms steht die mehrschichtige Backend-Architektur bestehend aus:

- Routing-Schicht,
- Controller-Schicht,
- Service-Schicht,
- Transformations- und Modellschicht.

Die `parkingRoutes.js` definiert dabei die verfügbaren API-Endpunkte und leitet eingehende Requests an den `parkingController.js` weiter.

Der Controller übernimmt die Validierung von Parametern sowie die Verarbeitung der HTTP-Anfragen und delegiert die eigentliche Geschäftslogik an den `parkingService.js`.

Die Service-Schicht bildet den Kern der Anwendung. Dort werden externe Parkplatzdaten geladen, verarbeitet, gefiltert und zwischengespeichert. Zusätzlich übernimmt der Service die Berechnung von Statistiken sowie die Verwaltung des Cache-Mechanismus.

Ein zentraler Bestandteil der Architektur ist der Transformationslayer (`transformer.js`). Dieser normalisiert die Daten unterschiedlicher externer APIs und überführt sie in ein einheitliches internes Datenmodell (`parkingModel.js`). Dadurch bleibt die Anwendung unabhängig von unterschiedlichen Datenformaten externer Anbieter.

Das Diagramm zeigt außerdem die Anbindung externer Dienste:

- eine externe Parkplatz-API zur Bereitstellung von Echtzeitdaten,
- sowie den Geocoding-Service auf Basis von Nominatim und OpenStreetMap zur Umwandlung von Ortsnamen in Koordinaten.

Zusätzlich wurde ein Fallback-Mechanismus über `db.js` integriert. Falls die externe API nicht erreichbar ist, können lokale Seed-Daten verwendet werden, wodurch die Stabilität der Anwendung erhöht wird.

6. Zentrale Entscheidungen

Im Verlauf der Umsetzung wurden mehrere grundlegende Architekturentscheidungen getroffen:

- Zentralisierung der Datenlogik im Backend,
- Einführung eines Transformationslayers,
- Nutzung einer klaren Schichtenarchitektur,
- Umsetzung eines ersten Cache-Konzepts.

Diese Entscheidungen verbessern insbesondere:

- Wartbarkeit,
- Erweiterbarkeit,
- Fehlerbehandlung,
- sowie die Stabilität der gesamten Anwendung.

Die Umsetzung zeigt, dass die gewählte Architektur grundsätzlich geeignet ist, die Anforderungen des Projekts zu erfüllen. Gleichzeitig wurde deutlich, dass die Verarbeitung von Echtzeitdaten deutlich komplexer ist als ursprünglich angenommen. Insbesondere die Datenqualität sowie die Performance externer APIs stellen Herausforderungen dar.

Der ursprünglich geplante direkte Zugriff des Frontends auf externe Datenquellen wurde daher bewusst verworfen. Stattdessen erfolgt die gesamte Datenverarbeitung zentral im Backend, wodurch Fehlerbehandlung, Datenvalidierung und Datenkonsistenz kontrollierter umgesetzt werden können.

7. Reflexion

Die Umsetzung zeigt, dass die gewählte Architektur grundsätzlich geeignet ist, die Anforderungen zu erfüllen. Insbesondere die Trennung zwischen Datenverarbeitung und Datenbereitstellung erweist sich als sinnvoll.

Gleichzeitig wurde deutlich, dass die Verarbeitung von Echtzeitdaten komplexer ist als initial angenommen. Der Transformationsprozess stellt aktuell den größten Engpass dar und beeinflusst sowohl Performance als auch Wartbarkeit.

Ein zentraler Schwachpunkt ist die bislang fehlende systematische Performanzanalyse. Ohne konkrete Messwerte ist eine gezielte Optimierung nur eingeschränkt möglich.

8. Zwischenstand und Ausblick

Das Backend ist funktional implementiert und bildet die Grundlage für die Anwendung.

Aktueller Stand:

- REST-API für Parkdaten
- Integration externer Datenquellen
- Transformationslogik
- Geocoding-Funktionalität
- Fallback-Mechanismen

Offene Schwerpunkte:

- Performanceanalyse und Optimierung
- Erweiterung der Cache-Strategie
- Reduktion der Transformationskomplexität
- Verbesserung der Datenverarbeitungseffizienz

Nächste Schritte:

- Einführung von Laufzeitmessungen zur Identifikation von Bottlenecks
- Optimierung der Transformationslogik
- Erweiterung des Caching-Konzepts
- Verlagerung von Berechnungen auf vorverarbeitete Daten

9. Fazit

Phase 2 markiert den Übergang von der Konzeptions- zur Implementierungsphase. Die geplante Architektur konnte erfolgreich umgesetzt werden und das Backend bildet nun die zentrale Grundlage der Anwendung.

Besonders die Trennung zwischen Frontend, Backend und externer API hat sich als sinnvoll erwiesen. Gleichzeitig wurde deutlich, dass Performance, Datenqualität und Stabilität entscheidende Herausforderungen für die weitere Entwicklung darstellen.

Insgesamt bildet der aktuelle Stand eine stabile Grundlage für die Finalisierung des Projekts in Phase 3.